

Code Layout, Readability and Reusability

Date: 15 July 2026

Team ID: XXXXXX

Project Name: Human Development Index (HDI) Prediction using Machine Learning

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation are maintained throughout the project.	Yes	Uses Python PEP-8 and consistent HTML/CSS formatting.
2	Proper File Structure	Project files are organized into logical folders.	Yes	Separate folders for Frontend, Backend, Models, Dataset, and Static files.
3	Meaningful Variable Names	Variables clearly describe their purpose.	Yes	Example: <code>life_expectancy,</code> <code>gni_per_capita,</code> <code>prediction_result.</code>
4	Function / Method Names	Functions have descriptive names.	Yes	Example: <code>predict_hdi(),</code> <code>load_dataset(),</code> <code>preprocess_data().</code>
5	Code Comments	Important logic is explained using comments.	Yes	Comments added for preprocessing, model training, and prediction.

6	Modular Design	Code is divided into reusable modules and functions.	Yes	Separate modules for preprocessing, training, prediction, and visualization.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	Common functionality reused through functions.
8	Error Handling	Exceptions are handled gracefully.	Yes	Input validation and exception handling implemented using <code>try-except</code> blocks.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Data Preprocessing Module	Python (Pandas, NumPy)	Dataset Cleaning, Feature Engineering	High
2	Machine Learning Prediction Module	Python (Scikit-learn)	HDI Prediction	High
3	Data Visualization Module	Matplotlib, Seaborn	Dashboard & Reports	High
4	Database Connection Module	MySQL Connector	User Data & Prediction History	Medium
5	Authentication Module	Flask	User Login & Session Management	Medium

Overall Code Quality Assessment

Aspect	Rating (1–5)	Comments
Code Layout & Structure	5	Well-organized folder structure with clear separation of frontend, backend, model, and database components.

Readability	5	Easy-to-understand code with meaningful naming conventions and comments.
Reusability	5	Modular functions allow reuse across multiple parts of the application.
Documentation / Comments	5	Functions and critical logic are documented for easy maintenance.
Overall Score	5 / 5	The project follows software engineering best practices and is maintainable, scalable, and reusable.

Suggested Project Folder Structure

```
HDI_Prediction_System/
├── app.py
├── requirements.txt
├── README.md
├── dataset/
│   └── hdi_dataset.csv
├── model/
│   ├── train_model.py
│   ├── predict_hdi.py
│   └── hdi_model.pkl
├── templates/
│   ├── index.html
│   └── dashboard.html
├── static/
│   ├── css/
│   ├── js/
│   └── images/
├── database/
│   └── database.sql
├── utils/
│   ├── preprocess.py
│   ├── visualization.py
│   └── helper.py
└── reports/
    └── prediction_reports/
```

Code Quality Summary

The **Human Development Index (HDI) Prediction System** follows industry-standard coding practices by maintaining a clean project structure, meaningful naming conventions, modular design, and proper documentation. Reusable modules for data preprocessing, machine learning prediction, visualization, authentication, and database operations improve maintainability and reduce code duplication. Error handling, consistent formatting, and clear documentation ensure the system is reliable, scalable, and easy to extend in future development.